

Support for Object-Oriented Programming

12.1 Introduction

12.2 Object-Oriented Programming

12.3 Design Issues for Object-Oriented Languages

12.4 Support for Object-Oriented Programming in Specific Languages

12.5 Implementation of Object-Oriented Constructs

12.6 Reflection

This chapter begins with a brief introduction to object-oriented programming, followed by an extended discussion of the primary design issues for inheritance and dynamic binding. Next, the support for object-oriented programming in Smalltalk, C++, Java, C#, and Ruby is discussed. Following this is a short overview of the implementation of dynamic bindings of method calls to methods in object-oriented languages. The last section discusses reflection.

12.1 Introduction

Languages that support object-oriented programming now are firmly entrenched in the mainstream. From COBOL to LISP, including virtually every language in between, dialects that support object-oriented programming have appeared. C++ supports procedural and data-oriented programming, in addition to object-oriented programming. CLOS, an object-oriented version of LISP (Paepeke, 1993), also supports functional programming. Some of the newer languages that were designed to support object-oriented programming do not support other programming paradigms but still employ some of the basic imperative structures and have the appearance of the older imperative languages. Among these are Java and C#. Ruby is challenging to categorize: It is a pure object-oriented language in the sense that all data are objects, but it is a hybrid language in that one can use it for procedural programming. Finally, there is the pure object-oriented but somewhat unconventional language: Smalltalk. Smalltalk was the first language to offer complete support for object-oriented programming. The details of support for object-oriented programming vary widely among languages, and that is the primary topic of this chapter.

This chapter relies heavily on Chapter 11. It is in fact a continuation of that chapter. This relationship reflects the reality that object-oriented programming is, in essence, an application of the principle of abstraction to abstract data types. Specifically, in object-oriented programming, the commonality of a collection of similar abstract data types is factored out and put in a new type. The members of the collection inherit these common parts from that new type. This feature is inheritance, which is at the center of object-oriented programming and the languages that support it.

The other characterizing feature of object-oriented programming, dynamic binding of method calls to methods, is also extensively discussed in this chapter.

Although object-oriented programming is supported by some of the functional languages, for example, CLOS, OCaml, and F#, those languages are not discussed in this chapter.

12.2 Object-Oriented Programming

12.2.1 Introduction

The concept of **object-oriented programming** had its roots in SIMULA 67 but was not fully developed¹ until the evolution of Smalltalk resulted in Smalltalk 80 (in 1980, of course). Indeed, some consider Smalltalk to be the base model for a purely object-oriented programming language. A language that is object oriented must provide support for three key language features: abstract data types, inheritance, and dynamic binding of method calls to methods. Abstract data types were discussed in detail in Chapter 11, so this chapter focuses on inheritance and dynamic binding.

12.2.2 Inheritance

There has long been pressure on software developers to increase their productivity. This pressure has been intensified by the continuing reduction in the cost of computer hardware. By the middle to late 1980s, it became apparent to many software developers that one of the most promising opportunities for increased productivity in their profession was in software reuse. Abstract data types, with their encapsulation and access controls, are obvious candidates for reuse. The problem with the reuse of abstract data types is that, in nearly all cases, the features and capabilities of the existing type are not quite right for the new use. The old type requires at least some minor modifications. Such modifications can be difficult, because they require the person doing the modification to understand part, if not all, of the existing code. In many cases, the person doing the modification is not the program's original author. Furthermore, the modifications often require changes to all client programs.

A second problem with programming with abstract data types is that the type definitions are all independent and are at the same level.² This design often makes it impossible to organize a program to match the problem space being addressed by the program. In many cases, the underlying problem has categories of objects that are related, both as siblings (being similar to each other) and as parents and children (having a descendant relationship).

Inheritance offers a solution to both the modification problem posed by abstract data type reuse and the program organization problem. If a new abstract data type can inherit the data and functionality of some existing type, and is also allowed to modify some of those entities and add new entities, reuse is greatly facilitated without requiring changes to the reused abstract data type. Programmers can begin with an existing abstract data type and design a modified descendant of it to fit a new problem requirement. Furthermore,

1. Although SIMULA 67 had classes, the members defined inside them were not hidden from outside code.
2. This is similar to the functions in a C program, which are also independent and at a single level.

inheritance provides a framework for the definition of hierarchies of related classes that can reflect the descendant relationships in the problem space.

The abstract data types in object-oriented languages, following the lead of SIMULA 67, are usually called **classes**. As with instances of abstract data types, class instances are called **objects**. A class that is defined through inheritance from another class is a **derived class**, a **subclass**, or a **child class**. A class from which the new class is derived is its **base class**, **superclass**, or **parent class**. The subprograms that define the operations on objects of a class are called **methods**. The calls to methods are sometimes called **messages**. The entire collection of methods of a class is called the **message protocol**, or **message interface**, of the class. Computations in an object-oriented program are specified by messages sent from objects to other objects, or in some cases, to classes.

Methods are similar to subprograms. Both are collections of code that perform some computation. Both can take parameters and return results.

Passing a message is different from calling a subprogram. A subprogram typically processes data that is either passed to it by its caller as a parameter or is accessed nonlocally or globally. A message that is sent to an object is a request to execute one of its methods. At least some of the data on which the method is to operate is part of the object itself. Objects have methods that define processes the object can perform on itself. Because the objects are of abstract data types, these should be the only ways to manipulate data of the object. A subprogram defines a process that it can perform on any data sent to it (or made available nonlocally or globally).

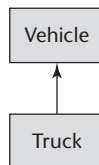
As a simple example of inheritance, consider the following: Suppose we have a class named `Vehicle`, which has variables for year, color, and make. A natural specialization, or subclass, of this would be `Truck`, which could inherit the variables from `Vehicle`, but would add variables for hauling capacity and number of wheels. Figure 12.1 shows a simple diagram to indicate the relationship between the `Vehicle` class and the `Truck` class, in which the arrow points to the parent class.

There are several ways a derived class can differ from its parent.³ Following are the most common differences between a parent class and its subclasses:

1. The subclass can add variables and/or methods to those inherited from the parent class.
2. The subclass can modify the behavior of one or more of its inherited methods. A modified method has the same name, and often the same protocol, as the one of which it is a modification.

Figure 12.1

A simple example of inheritance



3. If a subclass does not differ from its parent, it obviously serves no purpose.

3. The parent class can define some of its variables or methods to have private access, which means they will not be visible in the subclass.

The new method is said to **override** the inherited method, which is then called an **overridden method**. The purpose of an overriding method is to provide an operation in the subclass that is similar to one in the parent class, but is customized for objects of the subclass. For example, a parent class, `Bird`, might have a `draw` method that draws a generic bird. A subclass of `Bird` named `Waterfowl` could override the `draw` method inherited from `Bird` to draw a generic waterfowl, perhaps a duck.

Classes can have two kinds of methods and two kinds of variables. The most commonly used methods and variables are called **instance methods** and **instance variables**. Every object of a class has its own set of instance variables, which store the object's state. The only difference between two objects of the same class is the state of their instance variables.⁴ For example, a class for cars might have instance variables for color, make, model, and year. Instance methods operate only on the objects of the class. **Class variables** belong to the class, rather than their objects, so there is only one copy for the class. For example, if we wanted to count the number of instances of a class, the counter could not be an instance variable—it would need to be a class variable. **Class methods** can perform operations on the class, and possibly also on the objects of the class. They can be called by prefixing their names with either the class name or a variable that references one of their instances. If a class defines a class method, that method can be called even if there are no instances of the class. A class method could be used to create an instance of the class.

If a new class is a subclass of a single parent class, then the derivation process is called **single inheritance**. If a class has more than one parent class, the process is called **multiple inheritance**. When a number of classes are related through single inheritance, their relationships to each other can be shown in a derivation tree. The class relationships in a multiple inheritance can be shown in a derivation graph. This is shown in Figure 12.5 in Section 12.4.2.2.

One disadvantage of inheritance as a means of increasing the possibility of reuse is that it creates dependencies among the classes in an inheritance hierarchy. This result works against one of the advantages of abstract data types, which is that they are independent of each other. Of course, not all abstract data types must be completely independent. But in general, the independence of abstract data types is one of their strongest positive characteristics. However, it may be difficult, if not impossible, to increase the reusability of abstract data types without creating dependencies among some of them. Furthermore, in many cases, the dependencies naturally mirror dependencies in the underlying problem space.

In Chapter 11 the access controls for variables and methods, together often called members, in a class are discussed. Private members are visible inside the class, while public members also are visible to clients of the class. Inheritance brings a new category of possible visibility, subclasses. Private members of

4. This is not true in Ruby, which allows different objects of the same class to differ in other ways.

a base class are not visible to subclasses, but public members are. The third level of accessibility, protected, allows members of a base class to be visible to subclasses, but not clients.

12.2.3 Dynamic Binding

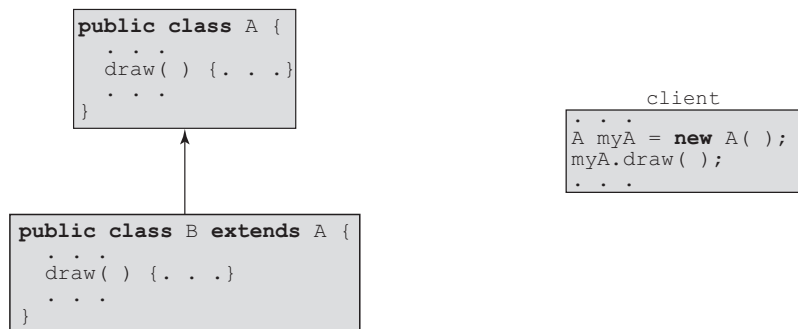
The third essential characteristic (after abstract data types and inheritance) of object-oriented programming languages is a kind of polymorphism⁵ provided by the dynamic binding of messages to method definitions. This is sometimes called **dynamic dispatch**. Consider the following situation: There is a base class, *A*, that defines a method `draw` that draws some figure associated with the base class. A second class, *B*, is defined as a subclass of *A*. Objects of this new class also need a `draw` method that is like that provided by *A* but a bit different because the subclass objects are slightly different. So, the subclass overrides the inherited `draw` method. If a client of *A* and *B* has a variable that is a reference to class *A*'s objects, that reference also could point at class *B*'s objects, making it a **polymorphic** reference. If the method `draw`, which is defined in both classes, is called through the polymorphic reference, the run-time system must determine, during execution, which method should be called, *A*'s or *B*'s (by determining which type object is currently referenced by the reference).⁶ Figure 12.2 shows this situation.

Polymorphism is a natural part of any object-oriented language that is statically typed. In a sense, polymorphism makes a statically typed language a little bit dynamically typed, where the little bit is in some bindings of method calls to methods. The type of a polymorphic variable is indeed dynamic.

One purpose of dynamic binding is to allow software systems to be more easily extended during both development and maintenance. Suppose we have a catalog of used cars that is implemented as a *Car* class and a subclass for each

Figure 12.2

Dynamic binding



5. Polymorphism is defined in Chapter 9.

6. Dynamic binding of method calls to methods is sometimes called *dynamic polymorphism*.

car in the catalog. The subclasses contain an image of the car and specific information about the car. Users can browse the cars with a program that displays the images and information about each car as the user browses to it. The display of each car (and its information) includes a button that the user can click if he or she is interested in that particular car. After the user gets through the catalog, the system will print the images and information about the cars of interest to the user. One way to implement this system is to place a reference to each car (subclass of `Car`) of interest in a list that can store references to the base class, `Car`. When the user is ready, information about all of the cars of interest could be printed for the user to study and compare the cars in the list. The catalog of cars will of course change frequently. This will necessitate corresponding changes in the subclasses of `Car`. However, changes to the collection of subclasses will not require any other changes to the system.

In some cases, the design of an inheritance hierarchy results in one or more classes that are so high in the hierarchy that an instantiation of them would not make sense. For example, suppose a program defined a `Building` class and a collection of subclasses for specific types of buildings, for instance, `French_Gothic_Cathedrals`. It probably would not make sense to have an implemented `draw` method in `Building`. But because all of its descendant classes should have such methods, the protocol (but not the body) of that method is included in `Building`. Such a method is often called an **abstract method** (*pure virtual method* in C++). A class that includes at least one abstract method is called an **abstract class** (*abstract base class* in C++). Such a class usually cannot be instantiated, because some of its methods are declared but are not defined (they do not have bodies). Any subclass of an abstract class that is to be instantiated must provide implementations (definitions) of all of the inherited abstract methods.

12.3 Design Issues for Object-Oriented Languages

A number of issues must be considered when designing the programming language features to support inheritance and dynamic binding. Those that we consider most important are discussed in this section.

12.3.1 The Exclusivity of Objects

A language designer who is totally committed to the object model of computation designs an object system that subsumes all other concepts of type. Everything, from a simple scalar integer to a complete software system, is an object in this mind-set. The advantage of this choice is the elegance and pure uniformity of the language and its use. The primary disadvantage is that simple operations must be done through the message-passing process, which often makes them slower than similar operations in an imperative model, where

single machine instructions may implement such simple operations. In this purest model of object-oriented computation, all types are classes. There is no distinction between predefined and user-defined classes. In fact, all classes are treated the same way and all computation is accomplished through message passing.

One alternative to the exclusive use of objects that is common in imperative languages to which support for object-oriented programming has been added is the following: Retain the complete collection of types from the base imperative language and add the object typing model. This approach results in a larger language whose type structure can be confusing to new users of the language.

Another alternative to the exclusive use of objects is to have an imperative-style type structure for the primitive scalar types, but implement all structured types as objects. This choice provides the speed of operations on primitive values that is comparable to those expected in the imperative model.

12.3.2 Are Subclasses Subtypes?

If a language allows programs in which a variable of a class can be substituted for a variable of one of its ancestor classes in any situation, without causing type errors and without changing the behavior of the program, that language supports the **principle of substitution**. In such a language, if class *B* is derived from class *A*, then *B* has everything *A* has and the behavior of an object of class *B*, when used in place of an object of class *A*, is identical to that of an object of class *A*.⁷ When this is true, *B* is a **subtype** of *A*. Although a subclass that is a subtype of its parent class must expose all of the members that are exposed by its parent class, the subclass can have members that are not in the parent class and still be a subtype.

The subtypes of Ada are examples of predefined subtypes. For example,

```
subtype Small_Int is Integer range -100..100;
```

Variables of `Small_Int` type have all of the operations of `Integer` variables but can store only a subset of the values possible in `Integer`. Furthermore, every `Small_Int` variable can be used anywhere an `Integer` variable can be used. That is, every `Small_Int` variable is, in a sense, an `Integer` variable.

The definition of subtype clearly disallows having public entities in the parent class that are not public in the subclass. So, the derivation process for subtypes must require that public entities of the parent class are inherited as public entities in the subclass.

Not all subclasses are subtypes and not all subtypes are subclasses. For example, a subclass cannot be a subtype if it changes the behavior of one of its overriding methods. Also, a class that is not a subclass of another class can be a subtype of that class by defining the same members, in terms of both types and

7. There is a fundamental theoretical problem with this last requirement: In general, it is not possible to determine if the behaviors of two programs are identical.

behavior. A subtype inherits interfaces and behavior, while a subclass inherits implementation, primarily to promote code reuse.

Most static-typed languages that support object-oriented programming are designed so that subclasses are subtypes, unless the programmer specifically designs a subclass that has behavior that differs from that of its parent class.

One obvious question is: Is the issue of whether subclasses are subtypes a theoretical or practical one? It is probably unusual to define a subclass whose overriding methods preserve the type protocols of their corresponding overridden methods but not their effects. So it is not a frequent practical issue. However, requiring all subclasses to be subtypes, if there were a reasonably simple way to enforce that, would place inheritance on a sounder theoretical base.

12.3.3 Single and Multiple Inheritance

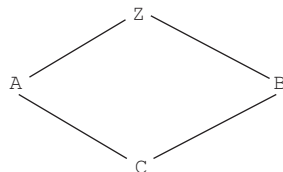
Another simple design issue for object-oriented languages is: Does the language allow multiple inheritance (in addition to single inheritance)? Maybe it's not so simple. The purpose of multiple inheritance is to allow a new class to inherit from two or more classes.

Because multiple inheritance is sometimes highly useful, why would a language designer not include it? The reasons lie in two categories: complexity and efficiency. The additional complexity is illustrated by several problems. First, note that if a class has two unrelated parent classes and neither defines a name that is defined in the other, there is no problem. However, suppose a subclass named *C* inherits from both class *A* and class *B* and both *A* and *B* define an inheritable method named *display*. If *C* needs to reference both versions of *display*, how can that be done? This ambiguity problem is further complicated when the two parent classes both define identically named methods and one or both of them must be overridden in the subclass.

Another issue arises if both *A* and *B* are derived from a common parent, *Z*, and *C* has both *A* and *B* as parent classes. This situation is called **diamond** or **shared** inheritance. In this case, both *A* and *B* should include *Z*'s inheritable variables. Suppose *Z* includes an inheritable variable named *sum*. The question is whether *C* should inherit both versions of *sum* or just one, and if just one, which one? There may be programming situations in which just one of the two should be inherited, and others in which both should be inherited. A similar problem occurs when both *A* and *B* inherit a method from *Z* and both override that method. If a client of *C*, which inherits both overriding methods, calls the method, which method is called, or are both supposed to be called. Diamond inheritance is shown in Figure 12.3.

Figure 12.3

An example of diamond inheritance



The question of efficiency may be more perceived than real. In C++, for example, supporting multiple inheritance requires just one additional array access and one extra addition operation for each dynamically bound method call, at least with some machine architectures (Stroustrup, 1994, p. 270). Although this operation is required even if the program does not use multiple inheritance, it is a small additional cost.

The use of multiple inheritance can easily lead to complex program organizations. Many who have attempted to use multiple inheritance have found that designing the classes to be used as multiple parents is difficult. And the difficulties are not restricted to those created by the initial developer. A class might be used by another developer at some later date as one of the parents of a new class. Maintenance of systems that use multiple inheritance can be a more serious problem, for multiple inheritance leads to more complex dependencies among classes. It is not clear to some that the benefits of multiple inheritance are worth the added effort to design and maintain a system that uses it.

An interface is somewhat similar to an abstract class; its methods are declared but not defined. Interfaces cannot be instantiated. They are used as an alternative to multiple inheritance.⁸ Interfaces provide some of the benefits of multiple inheritance but have fewer disadvantages. For example, the problems of diamond inheritance are avoided when interfaces, rather than multiple inheritance, are used.

12.3.4 Allocation and Deallocation of Objects

There are two design questions concerning the allocation and deallocation of objects. The first of these is the place from which objects are allocated. If they behave like the abstract data types, then they can be allocated from anywhere. This means they could be allocated from the run-time stack or explicitly created on the heap with an operator or function, such as `new`. If they are all heap dynamic, there is the advantage of having a uniform method of creation and access through pointer or reference variables. This design simplifies the assignment operation for objects, making it in all cases only a pointer or reference value change. It also allows references to objects to be implicitly dereferenced, simplifying the access syntax.

If objects are stack dynamic, there is a potential problem with regard to subtypes. If class `B` is a child of class `A` and `B` is a subtype of `A`, then an object of `B` type can be assigned to a variable of `A` type. For example, if `b1` is a variable of `B` type and `a1` is a variable of `A` type, then

```
a1 = b1;
```

is a legal statement. If `a1` and `b1` are references to heap-dynamic objects, there is no problem—the assignment is a simple pointer assignment. However, if `a1` and `b1` are stack dynamic, then they are value variables and, if assigned the

8. Interfaces initially appeared in Java, whose designers recognized the difficulties that are raised by the use of multiple inheritance.

value of the object, must be copied to the space of the target object. If *B* adds a data field to what it inherited from *A*, then *a1* will not have sufficient space on the stack for all of *b1*. The excess will simply be truncated, which could be confusing to programmers who write or use the code. This truncation is called **object slicing**. The following example and Figure 12.4 illustrate the problem.

```
class A {
    int x;
    . . .
};
class B : A {
    int y;
    . . .
}
```

The second question here is concerned with those cases where objects are allocated from the heap. The question is whether deallocation is implicit, explicit, or both. If deallocation is implicit, some implicit method of storage reclamation is required. If deallocation can be explicit, that raises the issue of whether dangling pointers or references can be created.

12.3.5 Dynamic and Static Binding

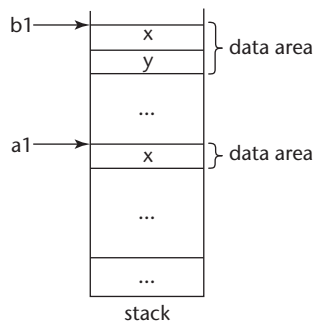
As discussed in Section 12.2.3, dynamic binding of messages to methods is an essential part of object-oriented programming. The question here is whether all bindings of messages to methods are dynamic. The alternative is to allow the user to specify whether a specific binding is to be dynamic or static. The advantage of this is that static bindings are faster. So, if a binding need not be dynamic, why pay the price?

12.3.6 Nested Classes

One of the primary motivations for nesting class definitions is information hiding. If a new class is needed by only one class, there is no reason to define it so it can be seen by other classes. In this situation, the new class can be nested

Figure 12.4

An example of object slicing



inside the class that uses it. In some cases, the new class is nested inside a sub-program, rather than directly in another class.

The class in which the new class is nested is called the **nesting class**. The most obvious design issues associated with class nesting are related to visibility. Specifically, one issue is: Which of the members of the nesting class are visible in the nested class? The other important issue is the opposite: Which of the members of the nested class are visible in the nesting class?

12.3.7 Initialization of Objects

The initialization issue is whether and how objects are initialized to values when they are created. This is more complicated than may be first thought. One question is whether objects must be initialized manually or through some implicit mechanism. When an object of a subclass is created, is the associated initialization of the inherited parent class member implicit or must the programmer explicitly deal with it?

12.4 Support for Object-Oriented Programming in Specific Languages

12.4.1 Smalltalk

Many think of Smalltalk as the definitive object-oriented programming language. It was the first language to include complete support for that paradigm. Therefore, it is natural to begin a survey of language support for object-oriented programming with Smalltalk.

12.4.1.1 General Characteristics

In Smalltalk, the concept of an object is truly universal. Virtually everything, from things as simple as the integer constant 2 to a complex file-handling system, is an object. As objects, they are treated uniformly. They all have local memory, inherent processing ability, the capability to communicate with other objects, and the possibility of inheriting methods and instance variables from ancestors. Classes cannot be nested in Smalltalk.

All computation is through messages, even a simple arithmetic operation. For example, the expression $x + 7$ is implemented as sending the $+$ message to x (to enact the $+$ method), sending 7 as the parameter. This operation returns a new numeric object with the result of the addition.

Replies to messages have the form of objects and are used to return requested or computed information or only to confirm that the requested service has been completed.

All Smalltalk objects are allocated from the heap and are referenced through reference variables, which are implicitly dereferenced. There is no

explicit deallocation statement or operation. So, all deallocation is implicit, using a garbage collection process for storage reclamation.

In Smalltalk, constructors must be explicitly called when an object is created. A class can have multiple constructors, but each must have a unique name.

Smalltalk classes cannot be nested in other classes.

Unlike a hybrid language such as C++, Smalltalk was designed for just one software development paradigm—object oriented. Furthermore, it adopts none of the appearance of the imperative languages. Its purity of purpose is reflected in its simple elegance and uniformity of design.

There is an example Smalltalk program in Chapter 2.

12.4.1.2 Inheritance

A Smalltalk subclass inherits all of the members of its superclass. The subclass can also have its own instance variables, which must have names that are distinct from the variable names in its ancestor classes. Finally, the subclass can define new methods and redefine methods that already exist in an ancestor class. When a subclass has a method whose name and protocol are the same as an ancestor class, the subclass method hides that of the ancestor class. Access to such a hidden method is provided by prefixing the message with the pseudovariable **super**. The prefix causes the method search to begin in the superclass rather than locally.

Because members in a parent class cannot be hidden from subclasses, subclasses can be and usually are subtypes.

Smalltalk does not support multiple inheritance.

12.4.1.3 Dynamic Binding

The dynamic binding of messages to methods in Smalltalk operates as follows: A message to an object causes a search of the class to which the object belongs for a corresponding method. If the search fails, it is continued in the superclass of that class, and so forth, up to the system class, `Object`, which has no superclass. `Object` is the root of the class derivation tree on which every class is a node. If no method is found anywhere in that chain, an error occurs. It is important to remember that this method search is dynamic—it takes place when the message is sent. Smalltalk does not, under any circumstances, bind messages to methods statically.

The only type checking in Smalltalk is dynamic, and the only type error occurs when a message is sent to an object that has no matching method, either locally or through inheritance. This is a different concept of type checking than that of most other languages. Smalltalk type checking has the simple goal of ensuring that a message matches some method.

Smalltalk variables are not typed; any name can be bound to any object. As a direct result, Smalltalk supports dynamic polymorphism. All Smalltalk code is generic in the sense that the types of the variables are irrelevant, as long as they are consistent. The meaning of an operation (method or operator) on a variable is determined by the class of the object to which the variable is currently bound.

The point of this discussion is that as long as the objects referenced in an expression have methods for the messages of the expression, the types of the objects are irrelevant. This means that no code is tied to a particular type.

12.4.1.4 Evaluation of Smalltalk

Smalltalk is a small language, although the Smalltalk system is large. The syntax of the language is simple and highly regular. It is a good example of the power that can be provided by a small language if that language is built around a simple but powerful concept. In the case of Smalltalk, that concept is that all programming can be done employing only a class hierarchy built using inheritance, objects, and message passing.

In comparison with conventional compiled imperative-language programs, equivalent Smalltalk programs are significantly slower. Although it is theoretically interesting that array indexing and loops can be provided within the message-passing model, efficiency is an important factor in the evaluation of programming languages. Therefore, efficiency will clearly be an issue in most discussions of the practical applicability of Smalltalk.

Smalltalk's dynamic binding allows type errors to go undetected until run time. A program can be written that includes messages to nonexistent methods and it will not be detected until the messages are sent, which causes a great deal more error repair later in the development than would occur in a static-typed language. However, in practice type errors are not a serious problem with Smalltalk programs.

Overall, the design of Smalltalk consistently came down on the side of language elegance and strict adherence to the principles of object-oriented programming support, often without regard for practical matters, in particular execution efficiency. This is most obvious in the exclusive use of objects and the typeless variables.

The Smalltalk user interface has had an important impact on computing: The integrated use of windows, mouse-pointing devices, and pop-up and pull-down menus, all of which first appeared in Smalltalk, dominate contemporary software systems.

Perhaps the greatest impact of Smalltalk is the advancement of object-oriented programming, now the most widely used design and coding methodology.

12.4.2 C++

Chapter 2 describes how C++ evolved from C and SIMULA 67, with the design goal of support for object-oriented programming while retaining nearly complete backward compatibility with C. C++ classes, as they are used to support abstract data types, are discussed in Chapter 11. C++ support for the other essentials of object-oriented programming is explored in this section. The whole collection of details of C++ classes, inheritance, and dynamic binding is large and complex. This section discusses only the most important among these topics, specifically, those directly related to the design issues described in Section 12.3.

C++ was the first widely used object-oriented programming language and is still among the most popular. So, naturally, it is the one with which other languages are often compared. For both of these reasons, our coverage of C++ here is more detailed than that of the other example languages discussed in this chapter.

12.4.2.1 General Characteristics

To maintain backward compatibility with C, C++ retains the type system of C and adds classes to it. Therefore, C++ has both traditional imperative-language types and the class structure of an object-oriented language. It supports methods, as well as functions that are not related to specific classes. This makes it a hybrid language, supporting both procedural programming and object-oriented programming.

The objects of C++ can be static, stack dynamic, or heap dynamic. Explicit deallocation using the **delete** operator is required for heap-dynamic objects, because C++ does not include implicit storage reclamation.

Many class definitions include a destructor method, which is implicitly called when an object of the class ceases to exist. The destructor is used to deallocate heap-allocated memory that is referenced by data members. It may also be used to record part or all of the state of the object just before it dies, usually for debugging purposes.

12.4.2.2 Inheritance

A C++ class can be derived from an existing class, which is then its parent, or base, class. Unlike Smalltalk and most other languages that support object-oriented programming, a C++ class can also be stand-alone, without a super-class. In the definition of a derived class, the name of the derived class has the name of the base class attached with a colon (:), as in the following syntactic form:

```
class derived_class_name : base_class_name { ... }
```

The data defined in a class definition are called *data members* of that class, and the functions defined in a class definition are called *member functions* of that class (member functions in other languages are usually called methods). Some or all of the members of the base class may be inherited by the derived class, which can also add new members and modify inherited member functions.

All C++ objects must be initialized before they are used. Therefore, all C++ classes include at least one constructor method that initializes the data members of the new object. Constructor methods are implicitly called when an object is created. If any of the data members are pointers to heap-allocated data, the constructor allocates that storage.

If a class is derived from another class, the inherited data members must be initialized when the derived class object is created. To do this, the base class constructor is implicitly called. When initialization data must be furnished to



On Paradigms and Better Programming

BJARNE STROUSTRUP

Bjarne Stroustrup is the designer and original implementer of C++ and the author of *A Tour of C++, Programming: Principles and Practice Using C++, The C++ Programming Language, The Design and Evolution of C++*, and many other publications. His research interests include distributed systems, design, programming techniques, software development tools, and programming languages. He is actively involved in the ANSI/ISO standardization of C++. Dr. Stroustrup is a managing director in the technology division of Morgan Stanley in New York City, a Visiting Professor in Computer Science at Columbia University, and a Distinguished Research Professor in Computer Science at Texas A&M University. He is a member of the National Academy of Engineering, an ACM Fellow, and an IEEE Fellow. In 1993, Stroustrup received the ACM Grace Murray Hopper Award “for his early work laying the foundations for the C++ programming language. Based on the foundations and Dr. Stroustrup’s continuing efforts, C++ has become one of the most influential programming languages in the history of computing.”

(year of interview: 2002)

PROGRAMMING PARADIGMS

Your thoughts on the object-oriented paradigm: Its pluses and minuses. Let me first say what I mean by OOP—too many people think that “object-oriented” is simply a synonym for “good.” If so, there would be no need for other paradigms. The key to OO is the use of class hierarchies providing polymorphic behavior through some rough equivalent of virtual functions. For proper OO, it is important to avoid directly accessing the data in such a hierarchy and to use only a well-designed functional interface.

In addition to its well-documented strengths, object-oriented programming also has obvious weaknesses. In particular, not every concept naturally fits into a class hierarchy, and the mechanisms supporting object-oriented programming can impose significant overheads compared to alternatives. For many simple abstractions, classes that do not rely on hierarchies and run-time binding provide a simpler and more efficient alternative. Furthermore, where no run-time resolution is needed, generic programming relying on (compile-time) parametric polymorphism is a better behaved and more efficient approach.

So, C++: Is it OO or other? C++ supports several paradigms—including OOP, generic programming, and procedural programming—and combinations of these paradigms define multiparadigm programming as supporting more than one programming style (“paradigm”) and combinations of those styles.

Do you have a mini-example of multiparadigm programming? Consider this variant of the classic “collection of shapes” examples (originating from the early days of the first language to support object-oriented programming: Simula 67):

```
void draw_all(const vector<Shape*>& vs)
{
    for (int i = 0; i<vs.size(); ++i)
        vs[i]->draw();
}
```

Here, I use the generic container `vector` together with the polymorphic type `Shape`. The `vector` provides static type safety and optimal run-time performance. The `Shape` provides the ability to handle

a Shape (i.e., any object of a class derived from Shape) without recompilation.

We can easily generalize this to any container that meets the C++ standard library requirements:

```
template<class C>
    void draw_all(const C& c)
{
    typedef typename C::
        const_iterator CI;
    for (CI p = c.begin();
         p!=c.end(); ++p)
        (*p)->draw();
}
```

Using iterators allows us to apply this draw_all () to containers that do not support subscripts, such as a standard library list:

```
vector<Shape*> vs;
list<Shape*> ls;
// . . .
draw_all(vs);
draw_all(ls);
```

We can even generalize this further to handle any sequence of elements defined by a pair of iterators:

```
template<class Iterator> void
draw_all(Iterator b, Iterator e)
{
    for_each(b,e,mem_fun(&Shape::draw));
}
```

To simplify the implementation, I used the standard library algorithm for_each.

We might call this last version of draw_all () for a standard library list and an array:

```
list<Shape*> ls;
Shape* as[100];
// . . .
draw_all(ls.begin(),ls.end());
draw_all(as,as+100);
```

SELECTING THE “RIGHT” LANGUAGE FOR THE JOB

How useful is it to have this background in numerous paradigms? Or would it be better to invest time in becoming even more familiar with OO languages rather than learning these other paradigms? It is essential for anyone who wants to be considered a professional in the areas of software to know several languages and several programming paradigms. Currently, C++ is the best language for multiparadigm programming and a good language for learning various forms of programming. However, it's not a good idea to know just C++, let alone to know just a single-paradigm language. That would be a bit like being colorblind or monoglot: You would hardly know what you were missing. Much of the inspiration to good programming comes from having learned and appreciated several programming styles and seen how they can be used in different languages.

Furthermore, I consider programming of any nontrivial program a job for professionals with a solid and broad education, rather than for people with a hurried and narrow “training.”

the base class constructor, it is given in the call to the derived object constructor. In general, this is done with the following construct:

```
subclass (subclass parameters) : base_class (superclass parameters) {
    ...
}
```

If no constructor is included by the developer in a class definition, the compiler includes a trivial constructor. This default constructor calls the constructor of the base class, if there is a base class.

Class members can be private, protected, or public. Private members are accessible only by member functions and friends of the class. Standalone functions, member functions, and classes can be declared to be friends of a class and thereby be given access to its private members. Public members are visible everywhere. Protected members are like private members, except in derived classes, whose access is described next. Derived classes can modify accessibility for their inherited members. The complete syntactic form of a derived class is as follows:

```
class derived_class_name : derivation_mode base_class_name
    { data member and member function declarations } ;
```

The `derivation_mode` can be either **public** or **private**.⁹ (Do not confuse public and private derivation with public and private members.) The public and protected members of a base class are also public and protected, respectively, in a public-derived class. In a private-derived class, both the public and protected members of the base class are private. So, in a class hierarchy, a private-derived class cuts off access to all members of all ancestor classes to all successor classes. Private members of a base class are inherited by a derived class, but they are not visible to the members of that derived class and are therefore of no use there. Private derivations provide the possibility that a subclass can have members with different access than the same members in the parent class. Consider the following example:

```
class base_class {
    private:
        int a;
        float x;
    protected:
        int b;
        float y;
    public:
        int c;
        float z;
};

class subclass_1 : public base_class { . . . };
class subclass_2 : private base_class { . . . };
```

9. It can also be **protected**, but that option is not discussed here.

In `subclass_1`, `b` and `y` are protected, and `c` and `z` are public. In `subclass_2`, `b`, `y`, `c`, and `z` are private. No derived class of `subclass_2` can have members with access to any member of `base_class`. The data members `a` and `x` in `base_class` are not accessible in either `subclass_1` or `subclass_2`.

Note that private-derived subclasses cannot be subtypes. For example, if the base class has a public data member, under private derivation that data member would be private in the subclass. Therefore, if an object of the subclass were substituted for an object of the base class, accesses to that data member would be illegal on the subclass object. However, public-derived subclasses can be and usually are subtypes.

Under private class derivation, no member of the parent class is implicitly visible to the instances of the derived class. Any member that must be made visible must be reexported in the derived class. This reexportation in effect exempts a member from being hidden even though the derivation was private. For example, consider the following class definition:

```
class subclass_3 : private base_class {
    base_class :: c;
    . . .
}
```

Now, instances of `subclass_3` can access `c`. As far as `c` is concerned, it is as if the derivation had been public. The double colon (`::`) in this class definition is a scope resolution operator. It specifies the class where its following entity is defined.

The example in the following paragraphs illustrates the purpose and use of private derivation.

Consider the following example of C++ inheritance, in which a general linked-list class is defined and then used to define two useful subclasses:

```
class single_linked_list {
private:
    class node {
        public:
            node *link;
            int contents;
    };
    node *head;
public:
    single_linked_list() {head = 0};
    void insert_at_head(int);
    void insert_at_tail(int);
    int remove_at_head();
    int empty();
};
```

The nested class, `node`, defines a cell of the linked list to consist of an integer variable and a pointer to a `node` object. The `node` class is in the private clause,

which hides it from all other classes. Its members are public, however, so they are visible to the nesting class, `single_linked_list`. If they were private, `node` would need to declare the nesting class to be a friend to make them visible in the nesting class. Note that nested classes have no special access to members of the nesting class. Only static data members of the nesting class are visible to methods of the nested class.¹⁰

The enclosing class, `single_linked_list`, has just a single data member, a pointer to act as the list's header. It contains a constructor function, which sets `head` to the null pointer value. The four member functions allow nodes to be inserted at either end of a list object, nodes to be removed from one end of a list, and lists to be tested for empty.

The following definitions provide stack and queue classes, both based on the `single_linked_list` class:

```
class stack : public single_linked_list {
public:
    stack() {}
    void push(int value) {
        insert_at_head(value);
    }
    int pop() {
        return remove_at_head();
    }
};

class queue : public single_linked_list {
public:
    queue() {}
    void enqueue(int value) {
        insert_at_tail(value);
    }
    int dequeue() {
        remove_at_head();
    }
};
```

Note that objects of both the `stack` and `queue` subclasses can access the `empty` function defined in the base class, `single_linked_list` (because it is a public derivation). Both subclasses define constructor functions that do nothing. When an object of a subclass is created, the proper constructor in the subclass is implicitly called. Then, any applicable constructor in the base class is called. So, in our example, when an object of type `stack` is created, the constructor in `single_linked_list` is called, which does the necessary initialization. Then the `stack` constructor is called, which does nothing.

The classes `stack` and `queue` both suffer from the same serious problem: Clients of both can access all of the public members of the parent class,

10. A class can also be defined in a method of a nesting class. The scope rules of such classes are the same as those for classes nested directly in other classes, even for the local variables declared in the method in which they are defined.

`single_linked_list`. A client of a `stack` object could call `insert_at_tail`, thereby destroying the integrity of its stack. Likewise, a client of a `queue` object could call `insert_at_head`. These unwanted accesses are allowed because both `stack` and `queue` are subtypes of `single_linked_list`. Public derivation is used when one wants the subclass to inherit the entire interface of the base class. The alternative is to use a derivation in which the subclass inherits only the implementation of the base class. Our two example derived classes can be written to make them not subtypes of their parent class by using **private**, rather than **public**, derivation.¹¹ Then, both will also need to reexport `empty`, because it will become hidden to their instances. This situation illustrates the motivation for the private-derivation option. The new definitions of the `stack` and `queue` types, named `stack_2` and `queue_2`, are shown in the following:

```
class stack_2 : private single_linked_list {
public:
    stack_2() {}
    void push(int value) {
        single_linked_list :: insert_at_head(value);
    }
    int pop() {
        return single_linked_list :: remove_at_head();
    }
    single_linked_list:: empty();
};
class queue_2 : private single_linked_list {
public:
    queue_2() {}
    void enqueue(int value) {
        single_linked_list :: insert_at_tail(value);
    }
    int dequeue() {
        single_linked_list :: remove_at_head();
    }
    single_linked_list:: empty();
};
```

These two classes use reexportation to allow access to base class methods for clients. This was not necessary when public derivation was used.

The two versions of `stack` and `queue` illustrate the difference between subtypes and derived types that are not subtypes. The linked list is a generalization of both stacks and queues, because both can be implemented as linked lists. So, it is natural to inherit from a linked-list class to define `stack` and `queue` classes. However, neither is a subtype of the linked-list class, because both make the public members of the parent class private, which makes them inaccessible to clients.

One of the reasons friends are necessary is that sometimes a subprogram must be written that can access the members of two different classes. For

11. They would not be subtypes because the public members of the parent class can be seen in a client, but not in a client of the subclass, where those members are private.

example, suppose a program uses a class for vectors and one for matrices, and a subprogram is needed to multiply a matrix object by a vector object. In C++, the multiply function can be made a friend of both classes.

As previously stated, C++ provides multiple inheritance. As an example, suppose we wanted a class for drawing that needed the behavior of a class written for drawing figures and the methods of the new class needed to run in a separate thread. We might define the following:

```
class Thread { . . . };  
class Drawing { . . . };  
class DrawThread : public Thread, public Drawing { . . . };
```

Class `DrawThread` inherits all of the members of both `Thread` and `Drawing`. If both `Thread` and `Drawing` happen to include members with the same name, they can be unambiguously referenced in objects of class `DrawThread` by using the scope resolution operator (`::`). This example of multiple inheritance is shown in Figure 12.5.

Some issues with the C++ implementation of multiple inheritance are discussed in Section 12.5.

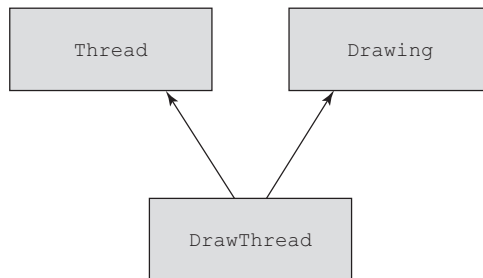
Overriding methods¹² in C++ must have exactly the same parameter profile as the overridden method. If there is any difference in the parameter profiles, the method in the subclass is considered a new method that is unrelated to the method with the same name in the ancestor class. The return type of the overriding method either must be the same as that of the overridden method or must be a publicly derived type of the return type of the overridden method.

12.4.2.3 Dynamic Binding

All of the member functions we have defined thus far are statically bound; that is, a call to one of them is statically bound to a function definition. A C++ object could be manipulated through a value variable, rather than a

Figure 12.5

Multiple inheritance



12. Recall that an overriding method is one that is defined in the derived class to replace a virtual method inherited from an ancestor class. Calls to an overriding method must be dynamically bound.

pointer or a reference. (Such an object would be static or stack dynamic.) However, in that case, the object's type is known and static, so dynamic binding is not needed. On the other hand, a pointer variable that has the type of a base class can be used to point to any heap-dynamic object of any class publicly derived from that base class, making it a polymorphic variable. Publicly derived subclasses are subtypes if none of the members of the base class are private. Privately derived subclasses are never subtypes. A pointer to a base class cannot be used to reference a method in a subclass that is not a subtype.

C++ does not allow value variables (as opposed to pointers or references) to be polymorphic. When a polymorphic variable is used to call a member function overridden in one of the derived classes, the call must be dynamically bound to the correct member function definition.

Consider the situation of having a base class named `Shape`, along with a collection of derived classes for different kinds of shapes, such as circles, rectangles, and so forth. If these shapes need to be displayed, then the displaying member function, `draw`, must be unique for each descendant, or kind of shape. These versions of `draw` must be defined to be virtual. When a call to `draw` is made with a pointer to the base class of the derived classes, that call must be dynamically bound to the member function of the correct derived class. The following example has the skeletal definitions for the example situation just described:

```
class Shape {
    public:
        virtual void draw() = 0;
        . . .
};
class Circle : public Shape {
    public:
        void draw() { . . . }
        . . .
};
class Rectangle : public Shape {
    public:
        void draw() { . . . }
        . . .
};
```

Given these definitions, the following code has examples of both statically and dynamically bound calls:

```
Circle* circ = new Circle;
Rectangle* rect = new Rectangle;
Shape* ptr_shape;
ptr_shape = circ;           // Now ptr_shape points to a
                             // Circle object
```


Any class that includes a pure virtual function is an **abstract class**. In C++, an abstract class is not marked with a reserved word. An abstract class can include completely defined methods. Because of the presence of one or more virtual functions, it is illegal to instantiate an abstract class. In a strict sense, an abstract class is one that is used only to represent the characteristics of a type. C++ provides abstract classes to model these truly abstract classes. If a subclass of an abstract class does not redefine a pure virtual function of its parent class, that function remains as a pure virtual function in the subclass and the subclass is also an abstract class.

Abstract classes and inheritance together support a powerful technique for software development. They allow types to be hierarchically defined so that related types can be subclasses of truly abstract types that define their common abstract characteristics.

Dynamic binding allows the code that uses members like `draw` to be written before all or even any of the versions of `draw` are written. New derived classes could be added years later, without requiring any change to the code that uses such dynamically bound members. This is a highly useful feature of object-oriented languages.

Reference assignments for stack-dynamic objects are different from pointer assignments for heap-dynamic objects. For example, consider the following code, which uses the same class hierarchy as the last example:

```
Circle circ;      // Allocate a Circle object on the stack
Rectangle rect;   // Allocate a Rectangle object on
                  // the stack
rect = circ;      // Copies the data member values from
                  // the Circle object
rect.draw();      // Calls the draw from the Rectangle
                  // object
```

In the assignment `rect = circ`, the member data from the object referenced by `circ` would be assigned to the data members of the object referenced by `rect`, but `rect` would still reference the `Rectangle` object. Therefore, the call to `draw` through the object referenced by `rect` would be that of the `Rectangle` class. If `rect` and `circ` were pointers to heap-dynamic objects, the same assignment would be a pointer assignment, which would make `rect` point to the `Circle` object, and a call to `draw` through `rect` would be bound dynamically to the `draw` in the `Circle` object.

12.4.2.4 Evaluation

It is natural to compare the object-oriented features of C++ with those of Smalltalk. The inheritance of C++ is more intricate than that of Smalltalk in terms of access control. By using both the access controls within the class definition and the derivation access controls, and also the possibility of friend

functions and friend classes, the C++ programmer has highly detailed control over access to class members. Although C++ provides multiple inheritance and Smalltalk does not, there are many who feel that is not an advantage for C++. The downsides of multiple inheritance weigh heavily against its value. In fact, C++ is the only language discussed in this chapter that supports multiple inheritance. On the other hand, languages that provide alternatives to multiple inheritance, such as Java and C#, clearly have an advantage over Smalltalk in that area.

In C++, the programmer can specify whether static binding or dynamic binding is to be used. Because static binding is faster, this is an advantage for those situations where dynamic binding is not necessary. Furthermore, even the dynamic binding in C++ is fast when compared with that of Smalltalk. Binding a virtual member function call in C++ to a function definition has a fixed cost, regardless of how distant in the inheritance hierarchy the definition appears. Calls to virtual functions require only five more memory references than statically bound calls (Stroustrup, 1988). In Smalltalk, however, messages are always dynamically bound to methods, and the farther away in the inheritance hierarchy the correct method is, the longer it takes. The disadvantage of allowing the user to decide which bindings are static and which are dynamic is that the original design must include these decisions, which may have to be changed later.

The static type checking of C++ is an advantage over Smalltalk, in which all type checking is dynamic. A Smalltalk program can be written with messages to nonexistent methods, which are not discovered until the program is executed. A C++ compiler finds such errors. Compiler-detected errors are less expensive to repair than those found in testing.

Smalltalk is essentially typeless, meaning that all code is effectively generic. This provides a great deal of flexibility, but static type checking is sacrificed. C++ provides generic classes through its template facility (as described in Chapter 11), which retains the benefits of static type checking.

The primary advantage of Smalltalk lies in the elegance and simplicity of the language, which results from the single philosophy of its design. It is purely and completely devoted to the object-oriented paradigm, devoid of compromises necessitated by the whims of an entrenched user base. C++, on the other hand, is a large and complex language with no single philosophy as its foundation, except to support object-oriented programming and include the C user base. One of its most significant goals was to preserve the efficiency and flavor of C while providing the advantages of object-oriented programming. Some people feel that the features of this language do not always fit well together and that at least some of the complexity is unnecessary.

According to Chambers and Ungar (1991), Smalltalk ran a particular set of small C-style benchmarks at only 10 percent of the speed of optimized C. C++ programs require only slightly more time than equivalent C programs (Stroustrup, 1988). Given the great efficiency gap between Smalltalk and C++,

it is little wonder that the commercial use of C++ is far more widespread than that of Smalltalk. There are other factors in this difference, but efficiency is clearly a strong argument in favor of C++. Of course, all of the compiled languages that support object-oriented programming run approximately 10 times faster than Smalltalk.

12.4.3 Java

Because Java's design of classes, inheritance, and methods is similar to that of C++, in this section we focus only on those areas in which Java differs from C++.

12.4.3.1 General Characteristics

As with C++, Java supports both objects and nonobject data. However, in Java, only values of the primitive scalar types (Boolean, character, and the numeric types) are not objects. Java's enumerations and arrays are objects. The reason Java has nonobjects is efficiency.

In Java 5.0+, primitive values are implicitly coerced when they are put in object context. This coercion converts the primitive value to an object of the wrapper class of the primitive value's type. For example, putting an `int` value or variable into object context causes the creation of an `Integer` object with the value of the `int` primitive. This coercion is called **boxing**.

Whereas C++ classes can be defined to have no parent, that is not possible in Java. All Java classes must be subclasses of the root class, `Object`, or some class that is a descendant of `Object`. One advantage of this is that some commonly needed methods, such as `toString` and `equals`, can be defined in `Object` and inherited and used by all other classes.

All Java objects are explicit heap dynamic. Most are allocated with the `new` operator, but there is no explicit deallocation operator. Garbage collection is used for storage reclamation. Like many other language features, although garbage collection avoids some serious problems, such as dangling pointers, it can cause other problems. One such difficulty arises because the garbage collector deallocates, or reclaims the storage occupied by an object, but it does no more. For example, if an object has access to some resource other than heap memory, such as a file or a lock on a shared resource, the garbage collector does not reclaim these. For these situations, Java allows the inclusion of a special method, `finalize`, which is related to a C++ destructor function.

A `finalize` method is implicitly called when the garbage collector is about to reclaim the storage occupied by the object. The problem with `finalize` is that the time it (and the garbage collector) will run cannot be forced or even predicted. The alternative to using `finalize` to reclaim resources held by an object about to be garbage collected is to include a method that does the reclamation. The only problem with this is that all clients of the objects must be aware of this method and remember to call it.

12.4.3.2 Inheritance

In Java, a method can be defined to be **final**, which means that it cannot be overridden in any descendant class. When the **final** reserved word is specified on a class definition, it means the class cannot be subclassed. All of the methods in a final class are implicitly final, which means that the bindings of method calls to the methods of the class are statically bound.

The advantage of defining a class to be final is that no changes to the class are allowed. For example, `String` is a final class and because of that any method that receives a `String` reference in a parameter can depend on the stability of the meaning of `String`'s methods. The disadvantage is that defining a class to be final disallows reuses that require even minor modifications.

Java includes the annotation `@Override`, which informs the compiler to check to determine whether the following method overrides a method in an ancestor class. If it does not, the compiler issues an error message.

Like C++, Java requires that parent class constructor be called before the subclass constructor is called. If parameters are to be passed to the parent class constructor, that constructor must be explicitly called, as in the following example:

```
super(100, true);
```

If there is no explicit call to the parent class constructor, the compiler inserts a call to the zero-parameter constructor in the parent class.

Java does not support the private derivations of C++. One can surmise that the Java designers believed that subclasses should be subtypes, which they are not when private derivations are supported. Thus, they did not include them. So, Java's subclasses can be subtypes.

Early versions of Java included a collection, `Vector`, which included a long list of methods for manipulating data in a collection construct. These versions of Java also included a subclass of `Vector`, `Stack`, which added methods for push and pop operations. Unfortunately, because Java does not have private derivation, all of the methods of `Vector` were also visible in the `Stack` class, which made `Stack` objects liable to a variety of operations that could invalidate those objects.

Java directly supports only single inheritance. However, it includes a kind of abstract class, called an **interface**, which provides partial support for multiple inheritance. An interface definition is similar to a class definition, except that it can contain only named constants and method declarations (not definitions). It cannot contain constructors, nonabstract methods, or variable declarations. So, an interface is no more than what its name indicates—it defines only the specification of a class. (Recall that a C++ abstract class can have instance variables and all but one of the methods can be completely defined.) A class does not inherit an interface; it implements it. In fact, a class can implement any number of interfaces. To implement an interface, the class must implement all of the methods whose specifications (but not bodies) appear in the interface definition.

An interface can be used to simulate multiple inheritance. A class can be derived from a class and implement an interface, with the interface taking the place of a second parent class. This is sometimes called **mixin inheritance**, because the constants and methods of the interface are mixed in with the methods and data inherited from the superclass, as well as any new data and/or methods defined in the subclass.

One more interesting capability of interfaces is that they provide another kind of polymorphism. This is because interfaces can be treated as types. For example, a method can specify a formal parameter that is an interface. Such a formal parameter can accept an actual parameter of any class that implements the interface, making the method polymorphic.

A nonparameter variable also can be declared to be of the type of an interface. Such a variable can reference any object of any class that implements the interface.

One of the problems with multiple inheritance occurs when a class is derived from two parent classes and both define a public method with the same name and protocol. This problem is avoided with interfaces. Although a class that implements an interface must provide definitions for all of the methods specified in the interface, if the class and the interface both include methods with the same name and protocol, the class need not reimplement that method. So, the method name conflicts that can occur with multiple inheritance cannot occur with single inheritance and interfaces. Furthermore, variable name conflicts are completely avoided because interfaces cannot define variables.

An interface is not a replacement for multiple inheritance, because in multiple inheritance there is code reuse, while interfaces provide no code reuse. This is an important difference, because code reuse is one of the primary benefits of inheritance. Java provides one way to partially avoid this deficiency. One of the implemented interfaces could be replaced by an abstract class, which could include code that could be inherited, thereby providing some code reuse.

One problem with interfaces being a replacement for multiple inheritance is the following: If a class attempts to implement two interfaces and both define methods that have the same name and protocol, there is no way to implement both in the class.

As an example of an interface, consider the `sort` method of the standard Java class, `Array`. Any class that uses this method must provide an implementation of a method to compare the elements to be sorted. The generic `Comparable` interface provides the protocol for this comparing method, which is named `compareTo`. The code for the `Comparable` interface is as follows:

```
public interface Comparable <T> {  
    public int compareTo(T b);  
}
```

The `compareTo` method must return a negative integer if the object through which it is called belongs before the parameter object, zero if they are equal, and a positive integer if the parameter belongs before the object through which `compareTo` was called. A class that implements the `Comparable`

interface can sort the contents of any array of objects of the generic type, as long as the implemented `compareTo` method for the generic type is implemented and provides the appropriate value. Interfaces have become a common substitute for multiple inheritance. Some form of interfaces are now part of C#, Swift, Ruby, and Ada.

In addition to interfaces, Java also supports abstract classes, similar to those of C++. The abstract methods of a Java abstract class are represented as just the method's header, which includes the **abstract** reserved word. The abstract class is also marked **abstract**. Of course, abstract classes cannot be instantiated.

Chapter 14 illustrates the use of interfaces in Java event handling.

12.4.3.3 Dynamic Binding

In C++, a method must be defined as virtual to allow dynamic binding. In Java, all method calls are dynamically bound unless the called method has been defined as **final**, in which case it cannot be overridden and all bindings are static. Static binding is also used if the method is **static** or **private**, both of which disallow overriding.

12.4.3.4 Nested Classes

Java has several varieties of nested classes, all of which have the advantage of being hidden from all classes in their package, except for the nesting class. Non-static classes that are nested directly in another class are called **inner classes**. Each instance of an inner class must have an implicit pointer to the instance of its nesting class to which it belongs. This gives the methods of the nested class access to all of the members of the nesting class, including the private members. Static nested classes do not have this pointer, so they cannot access members of the nesting class. Therefore, static nested classes in Java are like the nested classes of C++.

Though it seems odd in a static-scoped language, the members of the inner class, even the private members, are accessible in the outer class. Such references must include the variable that references the inner class object. For example, suppose the outer class creates an instance of the inner class with the following statement:

```
myInner = this.new Inner();
```

Then, if the inner class defines a variable named `sum`, it can be referenced in the outer class as `myInner.sum`.

An instance of a nested class can only exist within an instance of its nesting class. Nested classes can also be anonymous. Anonymous nested classes have complex syntax but are really only an abbreviated way to define a class that is used from just one location. An example of an anonymous nested class appears in Chapter 14.

A **local nested class** is defined in a method of its nesting class. Local nested classes are never defined with an access specifier (**private** or **public**). Their scope is always limited to their nesting class. A method in a local nested class can access the variables defined in its nesting class and the **final** variables defined in the method in which the local nested class is defined. The members of a local nested class are visible only in the method in which the local nested class is defined.

12.4.3.5 Evaluation

Java's design for supporting object-oriented programming is similar to that of C++, but it employs more consistent adherence to object-oriented principles. Java does not allow parentless classes and uses dynamic binding as the "normal" way to bind method calls to method definitions. This, of course, increases execution time slightly over languages in which many method bindings are static. At the time this design decision was made, however, most Java programs were interpreted, so interpretation time made the extra binding time insignificant. Access controls for the contents of a class definition are rather simple when compared with the jungle of access controls of C++, ranging from derivation controls to friend functions. Finally, Java uses interfaces to provide a form of support for multiple inheritance, which does not have all of the drawbacks of actual multiple inheritance.

12.4.4 C#

C#'s support for object-oriented programming is similar to that of Java.

12.4.4.1 General Characteristics

C# includes both classes and structs, with the classes being very similar to Java's classes and the structs being somewhat less powerful constructs. One important difference is that structs are value types; that is, they are stack dynamic. This could cause the problem of object slicing, but this is prevented by the restriction that structs cannot be subclassed. More details of how C# structs differ from its classes appeared in Chapter 11.

12.4.4.2 Inheritance

C# uses the syntax of C++ for defining classes. For example,

```
public class NewClass : ParentClass { . . . }
```

A method inherited from the parent class can be replaced in the derived class by marking its definition in the subclass with **new**. The **new** method hides the method of the same name in the parent class to normal access. However,

the parent class version can still be called by prefixing the call with **base**. For example,

```
base.Draw();
```

As with Java, subclasses can be subtypes. C#'s support for interfaces is the same as that of Java. It does not support multiple inheritance.

12.4.4.3 Dynamic Binding

To allow dynamic binding of method calls to methods in C#, both the base method and its corresponding methods in derived classes must be specially marked. The base class method must be marked with **virtual**, as in C++. To make clear the intent of a method in a subclass that has the same name and protocol as a virtual method in an ancestor class, C# requires that such methods be marked **override** if they are to override the parent class virtual method.¹³ For example, the C# version of the C++ `Shape` class that appears in Section 12.4.2.3 is as follows:

```
public class Shape {
    public virtual void Draw() { . . . }
    . . .
}
public class Circle : Shape {
    public override void Draw() { . . . }
    . . .
}
public class Rectangle : Shape {
    public override void Draw() { . . . }
    . . .
}
public class Square : Rectangle {
    public override void Draw() { . . . }
    . . .
}
```

C# includes abstract methods similar to those of C++, except that they are specified with different syntax. For example, the following is a C# abstract method:

```
abstract public void Draw();
```

A class that includes at least one abstract method is an abstract class, and every abstract class must be marked **abstract**. Abstract classes cannot be

13. Recall that this can be specified in Java with the annotation `@Override`.

instantiated. It follows that any subclass of an abstract class that will be instantiated must implement all abstract methods that it inherits.

As with Java, all C# classes are ultimately derived from a single root class, `Object`. The `Object` class defines a collection of methods, including `ToString`, `Finalize`, and `Equals`, which are inherited by all C# types.

12.4.4.4 Nested Classes

A C# class that is directly nested in a class behaves like a Java static nested class (which is like a nested class in C++). Like C++, C# does not support nested classes that behave like the nonstatic nested classes of Java.

12.4.4.5 Evaluation

Because C# is a recently designed C-based object-oriented language, one should expect that its designers learned from their predecessors and duplicated the successes of the past and remedied some of the problems. One result of this, coupled with the few problems with Java, is that the differences between C#'s support for object-oriented programming and that of Java are relatively minor. The availability of structs in C#, which Java does not have, can be considered an improvement. Like that of Java, C#'s support for object-oriented programming is simpler than that of C++, which many consider an improvement.

12.4.5 Ruby

As stated previously, Ruby is a pure object-oriented programming language in the sense of Smalltalk. Virtually everything in the language is an object and all computation is accomplished through message passing. Although programs have expressions that use infix operators and therefore have the same appearance as expressions in languages like Java, those expressions actually are evaluated through message passing. As is the case with Smalltalk, when one writes `a + b`, it is evaluated by sending the message `+` to the object referenced by `a`, passing a reference to the object `b` as a parameter. In other words, `a + b` is implemented as `a.+ b`.

12.4.5.1 General Characteristics

Ruby class definitions differ from those of languages such as C++ and Java in that they are executable. Because of this, they are allowed to remain open during execution. A program can add members to a class any number of times, simply by providing secondary definitions of the class that include the new members. During execution, the current definition of a class is the union of all definitions of the class that have been executed. Method definitions are also executable, which allows a program to choose between two versions of a method definition during execution, simply by putting the two definitions in the `then` and `else` clause of a selection construct.

Ruby objects are created with **new**, which implicitly calls a constructor. The usual constructor in a Ruby class is named `initialize`. A constructor in a subclass can initialize the data members of the parent class that have setters defined. This is done by calling **super** with the initial values as actual parameters. **super** calls the method in the parent class that has the same name as the method in which the call to **super** appears.

Ruby classes can be nested, but the nested class has no special access to the variables or methods of the nesting class.

All variables in Ruby are references to objects, and all are typeless. Recall that the names of all instance variables in Ruby begin with an at sign (@).

In a clear departure from the other common programming languages, access control in Ruby is different for data than it is for methods. All instance data has private access by default, and that cannot be changed. Therefore, no subclass in Ruby is a subtype. If external access to an instance variable is required, accessor methods must be defined. For example, consider the following skeletal class definition:

```
class MyClass

  # A constructor
  def initialize
    @one = 1
    @two = 2
  end

  # A getter for @one
  def one
    @one
  end

  # A setter for @one
  def one=(my_one)
    @one = my_one
  end

end # of class MyClass
```

The equal sign (=) attached to the name of the setter method means that its variable is assignable. So, all setter methods have equal signs attached to their names. The body of the `one` getter method illustrates the Ruby design of methods returning the value of the last expression evaluated when there is no return statement. In this case, the value of `@one` is returned.

Because getter and setter methods are so frequently needed, Ruby provides shortcuts for creating them. If one wants a class to have getter methods for the two instance variables, `@one` and `@two`, those getters can be specified with the single statement in the class:

```
attr_reader :one, :two
```

`attr_reader` is actually a function call, using `:one` and `:two` as the actual parameters. Preceding a variable with a colon (:) causes the variable name to

be used, rather than dereferencing it to the object to which it refers. Instead of passing a value or an address, the text of the variable's name is passed. This is exactly how macro parameters are passed.

The function that similarly creates setters is called `attr_writer`. This function has the same parameter profile as `attr_reader`.

The functions for creating getter and setter methods are so named because they provide the protocol for objects of the class, which then are called **attributes**. So, the attributes of a class define the data interface (the data made public through accessor methods) to objects of the class.

Class variables, which are specified by preceding their names with two at signs (`@@`), are private to the class and its instances. That privacy cannot be changed. Also, unlike global and instance variables, class variables must be initialized before they are used.

12.4.5.2 Inheritance

Subclasses are defined in Ruby using the less-than symbol (`<`), rather than the colon of C++. For example,

```
class MySubClass < BaseClass
```

One distinct thing about the method access controls of Ruby is that they can be changed in a subclass, simply by calling the access control functions. This means that two subclasses of a base class can be defined so that objects of one of the subclasses can access a method defined in the base class, but objects of the other subclass cannot. Also, this allows one to change the access of a publicly accessible method in the base class to a privately accessible method in the subclass.

12.4.5.3 Dynamic Binding

Support for dynamic binding in Ruby is the same as it is in Smalltalk. Variables are not typed; rather, they are all references to objects of any class. So, all variables are polymorphic and all bindings of method calls to methods are dynamic.

12.4.5.4 Evaluation

Because Ruby is an object-oriented programming language in the purest sense, its support for object-oriented programming is obviously adequate. However, access control to class members is weaker than that of C++. Ruby does not support abstract classes or interfaces, although its mixins are closely related to interfaces. Finally, in large part because Ruby is interpreted, its execution efficiency is far worse than that of the compiled languages.

Table 12.1 summarizes how the designers of the languages in this section chose to deal with the design issues described in Section 12.3.

Table 12.1 Designs

DESIGN ISSUE/ LANGUAGE	SMALLTALK	C++	JAVA	C#	RUBY
Exclusivity of objects	All data are objects	Primitive types plus objects	Primitive types plus objects	Primitive types plus objects	All data are objects
Are subclasses subtypes?	They can be and usually are	They can be and usually are if the derivation is public	They can be and usually are	They can be and usually are	No subclasses are subtypes
Single and multiple inheritance	Single only	Both	Single only, but some effects with interfaces	Single only, but some effects with interfaces	Single only, but some effects with modules
Allocation and deallocation of objects	All objects are heap allocated; allocation is explicit and deallocation is implicit	Objects can be static, stack dynamic, or heap dynamic; allocation and deallocation are explicit	All objects are heap dynamic; allocation is explicit and deallocation is implicit	All objects are heap dynamic; allocation is explicit and deallocation is implicit	All objects are heap dynamic; allocation is explicit and deallocation is implicit
Dynamic and static binding	All method bindings are dynamic	Method binding can be either	Method binding can be either	Method binding can be either	All method bindings are dynamic
Nested classes?	No	Yes	Yes	Yes	Yes
Initialization	Constructors must be explicitly called	Constructors are implicitly called	Constructors are implicitly called	Constructors are implicitly called	Constructors are implicitly called

12.5 Implementation of Object-Oriented Constructs

There are at least two parts of language support for object-oriented programming that pose interesting questions for language implementers: storage structures for instance variables and the dynamic bindings of messages to methods. In this section, we provide a brief look at these.

12.5.1 Instance Data Storage

In C++, classes are defined as extensions of C's record structures—structs. This similarity suggests a storage structure for the instance variables of class instances—that of a record. This form of this structure is called a **class instance record (CIR)**. The structure of a CIR is static, so it is built at compile time and used as a template for the creation of the data of class instances. Every class has its own CIR. When a derivation takes place, the CIR for the subclass is a copy of that of the parent class, with entries for the new instance variables added at the end.

Because the structure of the CIR is static, access to all instance variables can be done as it is in records, using constant offsets from the beginning of the CIR instance. This makes these accesses as efficient as those for the fields of records.

12.5.2 Dynamic Binding of Method Calls to Methods

Methods in a class that are statically bound need not be involved in the CIR for the class. However, methods that will be dynamically bound must have entries in this structure. Such entries could simply have a pointer to the code of the method, which must be set at object creation time. Calls to a method could then be connected to the corresponding code through this pointer in the CIR. The drawback to this technique is that every instance would need to store pointers to all dynamically bound methods that could be called from the instance.

Notice that the list of dynamically bound methods that can be called from an instance of a class is the same for all instances of that class. Therefore, the list of such methods must be stored only once. So the CIR for an instance needs only a single pointer to that list to enable it to find called methods. The storage structure for the list is often called a **virtual method table (vtable)**. Method calls can be represented as offsets from the beginning of the vtable. Polymorphic variables of an ancestor class always reference the CIR of the correct type object, so getting to the correct version of a dynamically bound method is assured. Consider the following Java example, in which all methods are dynamically bound:

```
public class A {  
    public int a, b;  
    public void draw() { . . . }  
    public int area() { . . . }  
}
```

```
public class B extends A {
    public int c, d;
    public void draw() { . . . }
    public void sift() { . . . }
}
```

The CIRs for the `A` and `B` classes, along with their vtables, are shown in Figure 12.7. Notice that the method pointer for the `area` method in `B`’s vtable points to the code for `A`’s `area` method. The reason is that `B` does not override `A`’s `area` method, so if a client of `B` calls `area`, it is the `area` method inherited from `A`. On the other hand, the pointers for `draw` and `sift` in `B`’s vtable point to `B`’s `draw` and `sift`. The `draw` method is overridden in `B` and `sift` is defined as an addition in `B`.

Multiple inheritance complicates the implementation of dynamic binding. Consider the following three C++ class definitions:

```
class A {
    public:
        int a;
        virtual void fun() { . . . }
        virtual void init() { . . . }
};
```

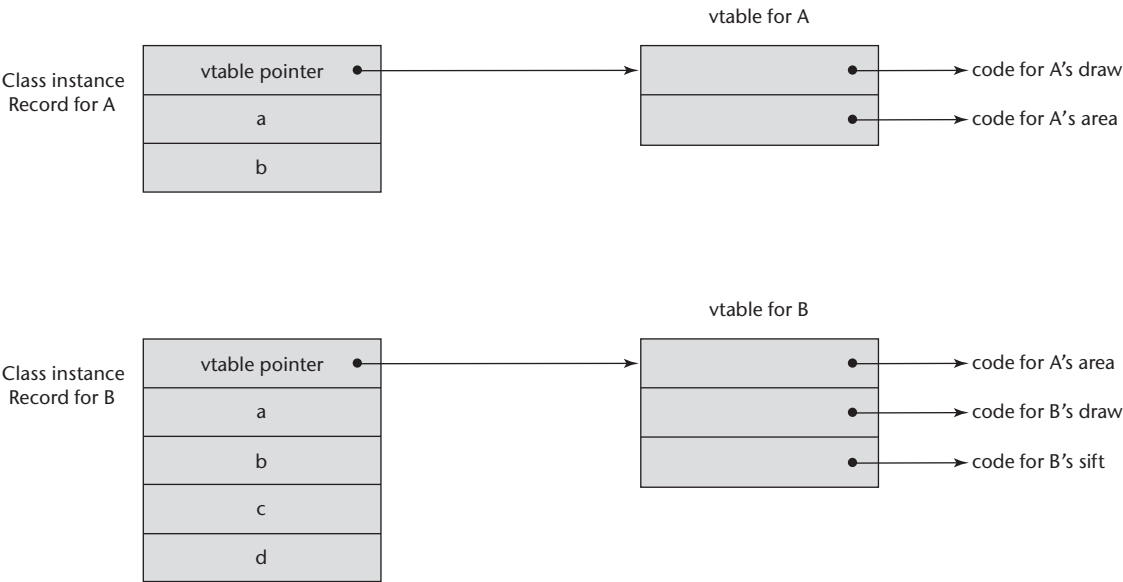


Figure 12.7
An example of the CIRs with single inheritance


```

class B {
    public:
        int b;
        virtual void sum() { . . . }
};
class C : public A, public B {
    public:
        int c;
        virtual void fun() { . . . }
        virtual void dud() { . . . }
};

```

The `C` class inherits the variable `a` and the `init` method from the `A` class. It redefines the `fun` method, although both its `fun` and that of the parent class `A` are potentially visible through a polymorphic variable (of type `A`). From `B`, `C` inherits the variable `b` and the `sum` method. `C` defines its own variable, `c`, and defines an uninherited method, `dud`. A CIR for `C` must include `A`'s data, `B`'s data, and `C`'s data, as well as some means of accessing all visible methods. Under single inheritance, the CIR would include a pointer to a vtable that has the addresses of the code of all visible methods. With multiple inheritance, however, it is not that simple. There must be at least two different views available in the CIR—one for each of the parent classes, one of which includes the view for the subclass, `C`. This inclusion of the view of the subclass in the parent class's view is just as in the implementation of single inheritance.

There must also be two vtables: one for the `A` and `C` view and one for the `B` view. The first part of the CIR for `C` in this case can be the `C` and `A` view, which begins with a vtable pointer for the methods of `C` and those inherited from `A`, and includes the data inherited from `A`. Following this in `C`'s CIR is the `B` view part, which begins with a vtable pointer for the virtual methods of `B`, which is followed by the data inherited from `B` and the data defined in `C`. The CIR for `C` is shown in Figure 12.8.

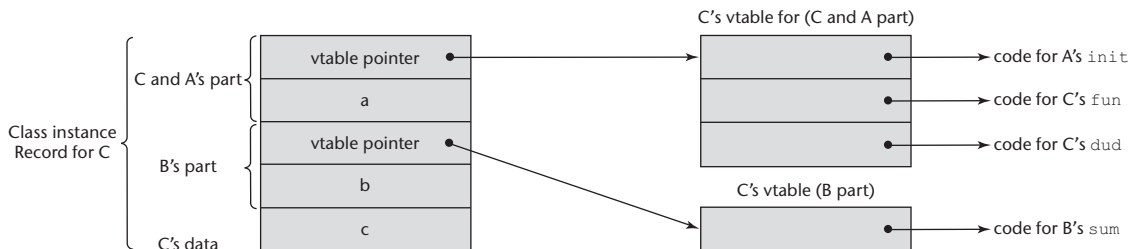


Figure 12.8

An example of a subclass CIR with multiple parents

12.6 Reflection

A discussion of reflection is not a perfect fit into a chapter on object orientation, but it is even a worse fit into any other chapter of this book. So, this is where we put it.

12.6.1 Introduction

In general, the later bindings take place in a programming language, the more flexible the language is. For example, the late binding of data types in scripting languages and functional languages allows their programs to be more generic than those in the static-typed languages. Likewise, the dynamic binding of method calls to methods that is part of the object-oriented languages allows their programs to be easier to maintain and extend. Among other things, reflection provides the possibility of late binding of calls to methods that are outside the inheritance hierarchy of the calling code.

12.6.2 What Is Reflection?

A programming language that supports reflection allows its programs to have run-time access to their types and structure and to be able to dynamically modify their behavior. To allow a program to examine its types and structure, that information must be gathered by the compiler or interpreter and made available to the program. Just as information about the structure of a database is called metadata, the types and structure of a program are called **metadata**. The process of a program examining its metadata is called **introspection**. A program can modify its behavior dynamically in several different ways: it could change its metadata directly, it could use the metadata, or it could intercede in the execution of the program. The first of these is complicated; the second is less complex and is common among languages; the third often is called **intercession**.

Some of the primary uses of reflection are in the construction of software tools. A class browser needs to enumerate the classes of a program. Visual Integrated Development Environments can use type information to assist a developer in building type-correct code. Debuggers must be able to examine private fields and methods of classes. Test systems need to be able to discover all of the methods of a class to be sure that test data drives all of them.

To illustrate a relatively simple and common use of reflection, we pose the following problem. A zoo has a large area devoted to birds. The flight cage for each species includes a plaque that provides general information about the inhabitant's species. Included on the plaque is a small screen onto which a visitor with special interest in the species can enter his or her entrance ticket number. At the exit to the bird exhibit, the visitor can again enter his or her ticket number on a small screen, which causes a computer to print pictures of the birds for which the visitor previously indicated particular interest. The computer system that supports these activities has an object that includes a method that draws a

picture of its bird for each of the birds on display. This seems simple enough. When a visitor selects a bird at its flight cage, the system places a reference to an object associated with that bird in a list. At the exit of the exhibit, the system calls the draw method of each object in the visitor's list. The process is complicated by the fact that the zoo supplies only some of the bird objects. Some of them are purchased from third-party vendors and some are donated by zoo benefactors. Because of the multiple sources of the bird objects, they do not have a common base class (other than `Object`) and do not implement a common interface, so what type references can be saved? One obvious solution is to make each bird object the subclass of a base class. References of the base class type could be stored in the list and dynamic binding could be used to invoke the draw methods. The drawback of this approach is that every bird class would need to be modified to make the new classes subclasses of the common base class. It would be better if the new bird classes could simply be added to a code file without modification. Another possible solution would be to use instance of and casting to determine the concrete types of the references. This would add much code to the system, increasing its complexity and cost of maintenance. A better solution is to use the dynamic binding that is possible with reflection.

12.6.3 Reflection in Java

Java provides limited support for reflection. The primary class of the metadata is defined in the namespace, `java.lang.Class`.¹⁴ This class has the unfortunately confusing name, `Class`. The Java run-time system instantiates an instance of `Class` for each object in the program. The `Class` class provides a collection of methods to examine the type information and members of the program objects. `Class` is the access point for all of the reflection API.

If the program has a reference to an object (not a primitive), the `Class` object of that object can be obtained by calling its `getClass` method. All classes inherit `getClass` from `Object`, from which all objects descend. Consider the following examples:

```
float[] totals = new float[100];
Class fltlist = totals.getClass();
Class stg = "hello".getClass();
```

The value of the variable `fltlist` will be the `Class` object of the `totals` array object. The value of `stg` will be the `Class` object of `String` (because `"hello"` is an instance of `String`).

If there is no object of a class, its `Class` object can be obtained through the class' name by attaching `.class` to the name. For example, we could have the following:

```
Class stg = String.class;
```

14. It is so named because its instantiations are themselves classes.

If the class has no name, its `Class` object can still be obtained by attaching `.class` to the class definition. For example, consider the following:

```
Class intmat = int[][].class;
```

The `.class` modifier can also be attached to primitive types. Although `float.getClass()` is illegal, `float.class` is not.

There are four methods to get the `Class` of a method. The `getMethod` method searches a class to find a specific public method defined in the class or inherited by the class. The `getMethods` method returns an array of all of the public methods defined in a class or inherited by the class. The `getDeclaredMethod` method searches for a specific method declared in a class, including private methods. The `getDeclaredMethods` method returns all of the methods defined in a class.

If the `Class` object of an object is known and a particular method defined by the class of the object is found, that method can be called through the `Method` object of the method with the `invoke` method. For example, if the `Method` object named `method` is found with `getMethod`, it can be called with the following:

```
method.invoke(...);
```

We can now develop a solution in Java for the problem posed in Section 12.6.2. The heart of this application is a class that defines a method that is passed an `Object` reference. The method determines the class of the passed reference, finds a `draw` method of that class, and calls that method. The solution class is tested with a second class, `ReflectTest`, which creates an array of three `Object` references to classes that represent three different birds. Each of these defines a `draw` method that, when called, displays a message indicating that it was called. Then the test calls the class method, passing the elements of the array of references.

The caller method can raise three different exceptions, each of which is handled in the method.

```
// A project to illustrate dynamic method calling
// using reflection in Java
package reflect;
import java.lang.reflect.*;
// A class to test the Reflect class
// Creates three objects that represent different birds
// and calls a method that dynamically calls the draw
// methods of the three bird classes
public class ReflectTest {
    public static void main(String[] args) {
        Object[] birdList = new Object[3];
        birdList[0] = new Bird1();
        birdList[1] = new Bird2();
        birdList[2] = new Bird3();
        Reflect.callDraw(birdList[2]);
        Reflect.callDraw(birdList[0]);
        Reflect.callDraw(birdList[1]);
    }
}
```

```

// A class to define the method that dynamically calls the
// methods of a passed class object
class Reflect {
    public static void callDraw(Object birdObj) {
        Class cls = birdObj.getClass();
        try {
            // Find the draw method of the given class
            Method method = cls.getMethod("draw");
            // Dynamically call the method
            method.invoke(birdObj);
        }
        // In case the given class does not support draw
        catch (NoSuchMethodException e) {
            throw new IllegalArgumentException (
                cls.getName() + "does not support draw");
        }
        // In case the callDraw cannot call draw
        catch (IllegalAccessException e) {
            throw new IllegalArgumentException (
                "Insufficient access permissions to call" +
                "draw in class " + cls.getName());
        }
        // In case draw throws an exception
        catch (InvocationTargetException e) {
            throw new RuntimeException(e);
        }
    }
}

class Bird1 {
    public void draw() {
        System.out.println("This is draw from Bird1");
    }
}

class Bird2 {
    public void draw() {
        System.out.println("This is draw from Bird2");
    }
}

class Bird3 {
    public void draw() {
        System.out.println("This is draw from Bird3");
    }
}
]

```

The output of this program is as follows:

```

This is the draw from Bird3
This is the draw from Bird1
This is the draw from Bird2

```

12.6.4 Reflection in C#

Support for reflection in C# is similar to that of Java, with a few important differences. In C#, as in all .NET languages, the compiler places the intermediate code, written in Common Intermediate Language (CIL), in an assembly, which could include several files. An assembly also contains an assembly version number and the metadata for all classes defined in the assembly, as well as for all external classes it uses.

Instead of the `java.lang.Class` namespace, `System.Type` is used in .NET; instead of `java.lang.reflect`, `System.Reflection` is used. Rather than the `getClass` method, `getType` is used to get the class of an instance. Also, the .NET languages use the `typeof` operator in place of the `.class` field used in Java. Following is a C# version of the Java project shown above:

```
using System;
using System.Reflection;
namespace TestReflect
{
    // A project to illustrate dynamic method calling
    // using reflection in C#
    // A class to test the Reflect class
    // Creates three objects that represent different birds
    // and calls a method that dynamically calls the draw
    // methods of the three bird classes
    public class ReflectTest {
        public static void Main(String[] args) {
            Object[] birdList = new Object[3];
            birdList[0] = new Bird1();
            birdList[1] = new Bird2();
            birdList[2] = new Bird3();
            Reflect.callDraw(birdList[2]);
            Reflect.callDraw(birdList[0]);
            Reflect.callDraw(birdList[1]);
        }
    }
    // A class to define the method that dynamically calls the
    // methods of a passed class object
    class Reflect {
        public static void callDraw(Object birdObj) {
            Type typ = birdObj.GetType();
            // Find the draw method of the given class
            MethodInfo method = typ.GetMethod("draw");
            // Dynamically call the method
            method.Invoke(birdObj, null);
        }
    }
    class Bird1 {
        public void draw() {
            Console.WriteLine("This is draw from Bird1");
        }
    }
}
```

```
class Bird2 {  
    public void draw() {  
        Console.WriteLine("This is draw from Bird2");  
    }  
}  
class Bird3 {  
    public void draw() {  
        Console.WriteLine("This is draw from Bird3");  
    }  
}  
}
```

Our simple example of dynamic method binding shows just one of the many uses of reflection.

In addition to the methods and fields of a class, the following program elements can be accessed with reflection in both Java and C#: class modifiers, such as `public`, `static`, and `final`, constructors, method parameter types, and implemented interfaces. Also, a description of the inheritance path of a class can be introspected. In C#, but not Java, the names of the formal parameters of methods can be discovered.

One significant difference between Java's reflection and that of C# is the `System.Reflection.Emit` namespace, which is part of .NET. This namespace provides the ability to create CIL code and an assembly to house that code. Java provides no such capability, although it can be done with tools from other suppliers.

Although reflection adds a variety of capabilities to the static-typed languages Java and C#, the user of reflection must be aware of its downsides:

- Performance nearly always suffers with the use of reflection. Resolving types, methods, and fields at run time are not part of the cost of running nonreflective code. Also, when types are dynamically resolved, some optimizations cannot be done on the code.
- Reflection exposes private fields and methods, which violate the rules of abstraction and information hiding, and also may result in unexpected side effects and adversely affect portability.
- Although the advantage of early type checking is widely accepted, the late binding that is possible with reflection obviously negates that advantage.
- Some reflective operations may not work when the code is run under a security manager, also making it non-portable. One such security environment is that of running applets. In most cases, if a problem can be solved without reflection, reflection should not be used.

Reflection is an integral part of most dynamically typed languages. In LISP, reflection is routinely used and the dynamic construction and execution of code is not uncommon. In other interpreted languages, such as JavaScript, Perl, and Python, the symbol table is kept during interpretation, providing all useful type information.

In Python, for example, the `type` method returns the type of a given value. For example, `type([7, 14, 21])` is `list`. The `isinstance` method returns

a Boolean value if its first parameter has the type named in its second parameter. For example, `isinstance(17, int)` returns `True`. The `callable` function is used to determine whether an expression returns a function object. The `dir` function returns the list of attributes, both data and methods, of its parameter object.

S U M M A R Y

Object-oriented programming is based on three fundamental concepts: abstract data types, inheritance, and dynamic binding. Object-oriented programming languages support the paradigm with classes, methods, objects, and message passing.

The discussion of object-oriented programming languages in this chapter revolves around seven design issues: exclusivity of objects, subclasses and subtypes, type checking and polymorphism, single and multiple inheritance, dynamic binding, explicit or implicit deallocation of objects, and nested classes.

Smalltalk is a pure object-oriented language—everything is an object and all computation is accomplished through message passing. All type checking and binding of messages to methods is dynamic, and all inheritance is single. Smalltalk has no explicit deallocation operation.

C++ provides support for data abstraction, inheritance, and optional dynamic binding of messages to methods, along with all of the conventional features of C. This means that it has two distinct type systems. C++ provides multiple inheritance and explicit object deallocation. It includes a variety of access controls for the entities in classes, some of which prevent subclasses from being subtypes. Both constructor and destructor methods can be included in classes; both are usually implicitly called.

While Smalltalk's dynamic type binding provides somewhat more programming flexibility than the hybrid language C++, it is far less efficient.

Unlike C++, Java is not a hybrid language; it is meant to support only object-oriented programming. Java has both primitive scalar types and classes. All objects are allocated from the heap and are accessed through reference variables. There is no explicit object deallocation operation—garbage collection is used. The only subprograms are methods, and they can be called only through objects or classes. Only single inheritance is directly supported, although a kind of multiple inheritance is possible using interfaces. All binding of messages to methods is dynamic, except in the case of methods that cannot be overridden. In addition to classes, Java includes packages as a second encapsulation construct.

C#, which is based on C++ and Java, supports object-oriented programming. Objects can be instantiated from either classes or structs. The struct objects are stack dynamic and do not support inheritance. Methods in a derived class can call the hidden methods of the parent class by including **base** on the method name. Methods that can be overridden must be marked **virtual**, and the overriding methods must be marked with **override**. All classes (and all primitives) are derived from `Object`.

Ruby is an object-oriented scripting language in which all data are objects. As with Smalltalk, all objects are heap allocated and all variables are typeless references to objects. All constructors are named `initialize`. All instance data are private, but getter and setter methods can be easily included. The collection of all instance variables for which access methods have been provided forms the public interface to the class. Such instance data are called attributes. Ruby classes are dynamic in the sense that they are executable and can be changed at any time. Ruby supports only single inheritance.

The instance variables of a class are stored in a CIR, the structure of which is static. Subclasses have their own CIRs, as well as the CIR of their parent class. Dynamic binding is supported with a virtual method table, which stores pointers to specific methods. Multiple inheritance greatly complicates the implementation of CIRs and virtual method tables.

Reflection is a process by which a program can access its classes and types and possibly dynamically change them to affect program behavior. One of the primary uses of reflection is in the construction of software tools, such as visual program construction tools, debuggers, and test systems. The class and type information, called metadata, is collected by the compiler or interpreter for the language. In Java, the class information, such as the methods of the class, are available in the `Class` object of a class. Support for reflection in C#, which is similar to that of Java, is available in the `System.Reflection` namespace.

REVIEW QUESTIONS

1. Describe the three characteristic features of object-oriented languages.
2. What is the difference between a class variable and an instance variable?
3. What is multiple inheritance?
4. What is a polymorphic variable?
5. What is an overriding method?
6. Describe a situation where dynamic binding is a great advantage over static binding.
7. What is a virtual method?
8. What is an abstract method? What is an abstract class?
9. Describe briefly the seven design issues used in this chapter for object-oriented languages.
10. What is a nesting class?
11. What is the message protocol of an object?
12. From where are Smalltalk objects allocated?
13. Explain how Smalltalk messages are bound to methods. When does this take place?
14. What type checking is done in Smalltalk? When does it take place?

15. What kind of inheritance, single or multiple, does Smalltalk support?
16. What are the two most important effects that Smalltalk has had on computing?
17. In essence, all Smalltalk variables are of a single type. What is that type?
18. From where can C++ objects be allocated?
19. How are C++ heap-allocated objects deallocated?
20. Are all C++ subclasses subtypes? If so, explain. If not, why not?
21. Under what circumstances is a C++ method call statically bound to a method?
22. What drawback is there to allowing designers to specify which methods can be statically bound?
23. What are the differences between private and public derivations in C++?
24. What is a **friend** function in C++?
25. What is a pure virtual function in C++?
26. How are parameters sent to a superclass's constructor in C++?
27. What is the single most important practical difference between Smalltalk and C++?
28. How is the type system of Java different from that of C++?
29. From where can Java objects be allocated?
30. What is boxing?
31. How are Java objects deallocated?
32. Are all Java subclasses subtypes?
33. How are superclass constructors called in Java?
34. Under what circumstances is a Java method call statically bound to a method?
35. In what way do overriding methods in C# syntactically differ from their counterparts in C++?
36. How can the parent version of an inherited method that is overridden in a subclass be called in that subclass in C#?
37. How does Ruby implement primitive types, such as those for integer and floating-point data?
38. How are getter methods defined in a Ruby class?
39. What access controls does Ruby support for instance variables?
40. What access controls does Ruby support for methods?
41. Are all Ruby subclasses subtypes?
42. Does Ruby support multiple inheritance?
43. What does reflection allow a program to do?
44. In the context of reflection, what is metadata?
45. What is introspection?

46. What is intercession?
47. What class in Java stores information about classes in a program?
48. For what is the Java name extension `.class` used?
49. What does the Java `getMethods` method do?
50. For what is the C# namespace `System.Reflection.Emit` used?

PROBLEM SET

1. What important part of support for object-oriented programming is missing in SIMULA 67?
2. Explain the principle of substitution.
3. Explain the ways subclasses can be created that are not subtypes.
4. Compare the dynamic binding of C++ and Java.
5. Compare the class entity access controls of C++ and Java.
6. Compare the multiple inheritance of C++ with that provided by interfaces in Java.
7. What is one programming situation where multiple inheritance has a significant advantage over interfaces?
8. Explain the two problems with abstract data types that are ameliorated by inheritance.
9. Describe the categories of changes that a subclass can make to its parent class.
10. Explain one disadvantage of inheritance.
11. Explain the advantages and disadvantages of having all values in a language be objects.
12. What exactly does it mean for a subclass to have an is-a relationship with its parent class?
13. Describe the issue of how closely the parameters of an overriding method must match those of the method it overrides.
14. Explain type checking in Smalltalk.
15. The designers of Java obviously thought it was not worth the additional efficiency of allowing any method to be statically bound, as is the case with C++. What are the arguments for and against the Java design?
16. What is the primary reason why all Java objects have a common ancestor?
17. What is the purpose of the `finalize` clause in Java?
18. What would be gained if Java allowed stack-dynamic objects as well as heap-dynamic objects? What would be the disadvantage of having both?
19. What are the differences between a C++ abstract class and a Java interface?

20. Explain why allowing a class to implement multiple interfaces in Java and C# does not create the same problems that multiple inheritance in C++ creates.
21. Study and explain the issue of why C# does not include Java's nonstatic nested classes.
22. Can you define a reference variable for an abstract class? What use would such a variable have?
23. Compare the access controls for instance variables in Java and Ruby.
24. Compare the type error detection for instance variables in Java and Ruby.
25. Explain the downsides of reflection.

PROGRAMMING EXERCISES

1. Rewrite the `single_linked_list`, `stack_2`, and `queue_2` classes in Section 12.5.2 in Java and compare the result with the C++ version in terms of readability and ease of programming.
2. Repeat Programming Exercise 1 using Ruby.
3. Design and implement a C++ program that defines a base class A, which has a subclass B, which itself has a subclass C. The A class must implement a method, which is overridden in both B and C. You must also write a test class that instantiates A, B, and C and includes three calls to the method. One of the calls must be statically bound to A's method. One call must be dynamically bound to B's method, and one must be dynamically bound to C's method. All of the method calls must be through a pointer to class A.
4. Write a program in C++ that calls both a dynamically bound method and a statically bound method a large number of times, timing the calls to both of the two. Compare the timing results and compute the difference of the time required by the two. Explain the results.
5. Repeat Programming Exercise 1 using Java, forcing static binding with **final**.